

Prompt Design — Concepts, Types & Best Practices (One-Cell Jupyter Markdown)

Prompts are simply the messages you send to an LLM. Every interaction with a model — a question, an instruction, a text to transform — is a **prompt**. Although we used many prompts in previous videos, calling them “prompts” makes their role explicit: the model’s output depends heavily on **how** you phrase the prompt.

1. Modalities of Prompts

Prompts are not limited to plain text. Modern models accept multimodal prompts such as:

- **Text** (most common)
- **Images** (upload an image and ask questions about it)
- **Audio** (supply an audio clip and ask for transcription/analysis)
- **Video** (ask questions about uploaded video content)

Today our focus is **text-based prompts**, because ~99% of real-world prompts are text for now.

2. Why Prompts Matter

- LLM outputs are **highly sensitive** to prompt wording. Small changes can produce large differences in answers (style, length, detail, accuracy).
 - Prompt design is central to reliable LLM behavior — so much so that a new practice and job role exists: **Prompt Engineering**.
-

3. Prompt Patterns & Techniques

Several practical prompt patterns are widely used. LangChain supports and encourages these patterns.

3.1 Static Prompts

- A **static prompt** is a fixed instruction string hard-coded in your app (e.g., `"Write a five-line poem about cricket."`).
- Static prompts are simple, but they do **not** adapt to user input or context. In production apps you typically avoid embedding fixed content directly in code.

3.2 Dynamic Prompts

- **Dynamic prompts** are templates with placeholders that get filled at runtime using user inputs or application data.
- Example template:
`Summarize {topic} in {tone}`
 - If user asks about `cricket` in `fun` tone → the placeholders are replaced and the filled prompt is sent to the LLM.
- Dynamic prompts are reusable and scalable for real applications.

3.3 Role-Based / System Prompts

- You can **assign a role** or persona to the model to shape behavior. Common pattern uses:
 - `system` message: high-level instruction ("You are an experienced doctor.")
 - `user` message: the user request
 - `assistant` message: (model responses)
- Role-based prompts make the model respond as a specialist (doctor, tutor, engineer, etc.) and are essential for domain-specific assistants.

3.4 Few-Shot / Example-Based Prompts

- **Few-shot prompting:** you provide several example input→output pairs to teach the model a task before giving the real query.
- Useful for classification, format-constrained outputs, or showing style/structure you expect.
- Example: show 3 labeled support tickets and categories → ask the model to categorize a new ticket.

3.5 Zero-Shot Prompts

- You describe the task without examples. Works well for well-specified instructions but less reliable than few-shot for edge cases.

4. Practical Prompt-Building Tips (Best Practices)

1. **Be explicit:** state the desired format, length, tone, and constraints (e.g., "Write a 3-line summary in plain English, no bullet points").
 2. **Use role/system messages** to set behavior for multi-turn or domain-specific applications.
 3. **Show examples** when you need structure or classification (few-shot).
 4. **Prefer templates** (dynamic prompts) over hard-coded strings — makes your app maintainable and reusable.
 5. **Constrain outputs** (max tokens, format) to control cost and predictability.
 6. **Test and iterate:** small wording changes can improve accuracy—treat prompt design as experimentation.
 7. **Validate outputs** programmatically when possible (parsing JSON, schema checks).
-

5. Production Considerations

- In real applications users provide inputs; you should build a dynamic prompt pipeline that:
 1. collects user inputs,
 2. fills a template (system + user parts),
 3. calls the model,
 4. post-processes and displays results.
 - Avoid writing prompts directly inside the model call at many places in your codebase. Maintain prompt templates centrally (files, templates engine, or prompt objects).
-

6. Prompt Engineering as a Discipline

- Prompt engineering blends product thinking + experimentation + domain knowledge.
 - It includes creating templates, curating examples, measuring model behavior, and adding guardrails (validation, safety filters).
 - LangChain provides reusable building blocks (prompt templates, role messages, chains) to make this engineering systematic.
-

7. Summary

- Prompts = the input you send to a model; small changes matter.
 - Use **dynamic templates**, **role/system prompts**, and **few-shot examples** to guide models reliably.
 - Implement prompts as maintainable assets, test them, and iterate — prompt engineering is a core skill for modern LLM apps.
-

If you want, next we can convert these prompt patterns into ready-to-use **LangChain prompt templates** and show common template examples you can plug into your app.

Static vs Dynamic Prompts (with Research Assistant Example)

◆ What Are We Building?

We consider an example where we build a **Research Assistant Tool**.
A user can open a webpage, type a prompt like:

“Summarize *Attention Is All You Need* in simple language.”

and the LLM returns a clean summary.

This workflow uses:

- **Streamlit** → to build UI
 - **LangChain / ChatOpenAI** → to call LLM
 - **dotenv** → to load API keys
-

1. Basic Flow of an LLM Application

1. User enters a prompt in the input box.
2. You send this prompt to the LLM.
3. LLM generates a response.
4. You display that response on the webpage.

A minimal Streamlit UI:

- `st.header()` → heading
- `st.text_input()` → user input
- `st.button()` → action
- `st.write()` → display output
- Run using → `streamlit run prompt_ui.py`

This is the basic workflow of any LLM-based application.

2. Static Prompts

When the user **types the full prompt manually**, it is called a **static prompt**.

Example:

Summarize the Word2Vec paper in five lines.

Problems with Static Prompts

Static prompts are **not recommended** for production apps. Why?

1. User's mistakes affect model output

- Wrong paper name
- Wrong format
- Typo
 - LLM output becomes wrong, hallucinated, or inconsistent.

2. Prompt sensitivity

LLMs are highly sensitive to small changes:

- “five lines” vs “maths-heavy summary”
- “simple explanation” vs “code-focused explanation”

Even tiny changes cause big output differences → inconsistent UX.

3. Loss of control

If the user writes the whole prompt, you cannot ensure:

- consistency
- quality
- structure
- specific instructions (e.g., include equations, analogies, or clarity)

Therefore static prompts create **unpredictable, low-quality experiences**.

```
In [7]: %%writefile static_prompt.py

from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
import streamlit as st
import os

load_dotenv(override=True)
api_key = os.getenv("OPEN_API_KEY")
model = ChatOpenAI(api_key=api_key)

st.header('Reasearch Tool')

user_input = st.text_input('Enter your prompt' )

if st.button("Summarize"):
    response = model.invoke(user_input)
    st.write(response.content)
```

Overwriting static_prompt.py

```
In [8]: !streamlit run static_prompt.py
```

^C

3. ♦ Why We Need Dynamic Prompts

Dynamic prompts solve all problems of static prompts.

How?

Instead of allowing the user to type the full prompt,
you create a fixed prompt template, like:

Please summarize the research paper titled: {paper} Use the following explanation style:
{style} Make the explanation of the following length: {length}

Include:

- Key mathematical equations (if present)
- Simple, intuitive code snippets (if applicable)
- Clear analogies
- Write "Insufficient information available" if some details are missing.

Ensure the summary is clear, accurate, and aligned with the requested style and length.

Here:

- `{paper}` → user-selected paper
- `{style}` → simple / maths-heavy / code-heavy
- `{length}` → short / medium / long

The template is **fixed**, only values are filled dynamically.

4. ♦ Upgrading the UI (Dynamic Inputs)

Instead of one text box, UI now has:

1. Dropdown for Paper Selection

- Avoids spelling errors
- Ensures correct paper name

2. Dropdown for Explanation Style

- simple
- maths-heavy
- code-heavy

3. Dropdown for Summary Length

- short
- medium
- long

These three values automatically fill the template.

5. ♦ What Is a Dynamic Prompt?

A **dynamic prompt** =
fixed template + **user-provided variables**.

You are in complete control:

- Structure
 - Requirements
 - Quality
- User only fills specific fields.

✓ Advantages

- Consistent outputs
- Controlled format
- No user mistakes
- Zero hallucination due to wrong paper name
- Better UX
- Reusable for any number of papers

This is why nearly all production LLM apps use **dynamic prompts**, not static ones.

✓ Final Understanding

Feature	Static Prompt	Dynamic Prompt
User types entire prompt	✓	X
Developer control	Low	High
Output consistency	Low	High
Error possibilities	High	Very low
Used in production apps?	Rarely	Always

Dynamic prompts make your LLM application **robust, reliable, and professional**.

prompt sample

Please summarize the research paper titled "{paper_input}" with the following specifications:

Explanation Style: {style_input}

Explanation Length: {length_input}

1. Mathematical Details:

- Include relevant mathematical equations if present in the paper.
- Explain the mathematical concepts using simple, intuitive code snippets where applicable.

2. Analogies:

- Use relatable analogies to simplify complex ideas.

If certain information is not available in the paper, respond with:

"Insufficient information available" instead of guessing.

Ensure the summary is clear, accurate, and aligned with the provided style and length.

```
In [13]: %%writefile dynamic_prompt.py

from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
import streamlit as st
from langchain_core.prompts import PromptTemplate, load_prompt
import os

load_dotenv(override=True)
api_key = os.getenv("OPEN_API_KEY")
model = ChatOpenAI(api_key=api_key)

st.header('Reasearch Tool')

paper_input = st.selectbox( "Select Research Paper Name", ["Attention Is All You
style_input = st.selectbox( "Select Explanation Style", ["Beginner-Friendly", "T
length_input = st.selectbox( "Select Explanation Length", ["Short (1-2 paragraph

template = PromptTemplate(
    template="""
        Please summarize the research paper titled "{paper_input}" with the
        Explanation Style: {style_input}
        Explanation Length: {length_input}
        1. Mathematical Details:
            - Include relevant mathematical equations if present in the paper
            - Explain the mathematical concepts using simple, intuitive code
        2. Analogies:
            - Use relatable analogies to simplify complex ideas.
        If certain information is not available in the paper, respond with:
        Ensure the summary is clear, accurate, and aligned with the provided
        """,
    input_variables=['paper_input', 'style_input', 'length_input']
)

# fill the placeholders
prompt = template.invoke({
    'paper_input' : paper_input,
    'style_input':style_input,
    'length_input':length_input
})

if st.button('Summarize'):
    result = model.invoke(prompt)
    st.write(result.content)
```

Overwriting dynamic_prompt.py

In [14]: ! streamlit run dynamic_prompt.py

^C

```
In [3]: %%writefile prompt_generator.py
from langchain_core.prompts import PromptTemplate

# template
template = PromptTemplate(
    template="""
Please summarize the research paper titled "{paper_input}" with the following sp
Explanation Style: {style_input}
Explanation Length: {length_input}
1. Mathematical Details:
    - Include relevant mathematical equations if present in the paper.
    - Explain the mathematical concepts using simple, intuitive code snippets whe
2. Analogies:
    - Use relatable analogies to simplify complex ideas.
If certain information is not available in the paper, respond with: "Insufficien
Ensure the summary is clear, accurate, and aligned with the provided style and l
""",
    input_variables=['paper_input', 'style_input', 'length_input'],
    validate_template=True
)

template.save('template.json')
```

Overwriting prompt_generator.py

```
In [6]: %%writefile load_template.py

from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
import streamlit as st
from langchain_core.prompts import PromptTemplate, load_prompt
import os

load_dotenv(override=True)
api_key = os.getenv("OPEN_API_KEY")
model = ChatOpenAI(api_key=api_key)

st.header('Reasearch Tool')

paper_input = st.selectbox( "Select Research Paper Name", ["Attention Is All You
style_input = st.selectbox( "Select Explanation Style", ["Beginner-Friendly", "T
length_input = st.selectbox( "Select Explanation Length", ["Short (1-2 paragraph
template = load_prompt('template.json')

if st.button('Summarize'):
    prompt = template.invoke({
        'paper_input': paper_input,
        'style_input': style_input,
```

```

        'length_input':length_input
    })

    result = model.invoke(prompt)
    st.write(result.content)

```

Overwriting load_template.py

In [7]: `!streamlit run load_template.py`

^C

In [1]: `%%writefile simple_prompt.py`

```

from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
import streamlit as st
from langchain_core.prompts import PromptTemplate, load_prompt
import os

load_dotenv(override=True)
api_key = os.getenv("OPEN_API_KEY")
model = ChatOpenAI(api_key=api_key)

st.header('Reasearch Tool')

paper_input = st.selectbox( "Select Research Paper Name", ["Attention Is All You
style_input = st.selectbox( "Select Explanation Style", ["Beginner-Friendly", "T
length_input = st.selectbox( "Select Explanation Length", ["Short (1-2 paragraph
template = load_prompt('template.json')

if st.button('Summarize'):
    chain = template | model
    result = chain.invoke({
        'paper_input':paper_input,
        'style_input':style_input,
        'length_input':length_input
    })
    st.write(result.content)

```

Writing simple_prompt.py

In [4]: `!streamlit run simple_prompt.py`

^C

Prompt Templates vs f-Strings — Conceptual Understanding

When learning **Prompt Templates**, a very natural doubt arises:

"If we are just inserting dynamic values inside a string, why not simply use Python f-strings?"

This is a **valid and important question**.

Yes — technically, **everything done using Prompt Templates can also be done using f-strings**.

But Prompt Templates exist for **strong architectural reasons**, not syntax convenience.

Below are the **core conceptual reasons** why Prompt Templates are preferred in real LLM applications.

1. Built-in Validation (Most Important Reason)

Prompt Templates provide **automatic validation** of placeholders.

What does validation mean?

- If your template expects certain variables (e.g. paper name, style, length),
- The system checks whether **all required inputs are provided**.
- If something is missing or extra, an **error is raised immediately**.

Why this matters:

- Errors are caught **during development**, not after deployment.
- Prevents silent bugs and broken prompts.
- Protects the application from incomplete or mismatched inputs.

Key takeaway:

- **Prompt Templates fail early and loudly**
- **f-strings fail late and silently**

This alone makes Prompt Templates safer for production systems.

2. Cleaner Structure & Readability

As prompts grow:

- They become long
- They include constraints, rules, formatting instructions
- They may span multiple paragraphs

Embedding large prompts directly inside application logic:

- Makes code bulky
- Reduces readability
- Mixes business logic with prompt logic

Prompt Templates:

- Separate **prompt design** from **application logic**
 - Keep code clean and easier to maintain
-

3. Reusability Across the Application

Prompt Templates are **designed to be reused**.

Real-world scenarios:

- Same prompt used across multiple pages
- Same template shared by multiple tools
- Prompt updated in one place, reflected everywhere

Templates can be:

- Stored separately (e.g. config or template files)
- Loaded wherever needed
- Version-controlled independently

With f-strings:

- Prompts often get duplicated
 - Changes must be manually updated in multiple places
 - Higher chance of inconsistencies
-

4. Scalability & Team Collaboration

In real projects:

- Prompt design may be handled by a different team
- Developers focus on application logic
- Prompt engineers focus on output quality

Prompt Templates enable:

- Clear contract between prompt inputs and outputs
 - Easier collaboration
 - Better prompt versioning and experimentation
-

5. Conceptual Difference (Mental Model)

Think of it like this:

- **f-strings** → Quick string interpolation tool

- **Prompt Templates** → Structured, validated prompt contracts

Prompt Templates treat prompts as **first-class citizens**, not just strings.

Final Summary

Aspect	f-Strings	Prompt Templates
Validation	❌ No	✅ Yes
Error detection	Runtime / Silent	Early / Explicit
Code cleanliness	Low	High
Reusability	Poor	Excellent
Production readiness	Weak	Strong

Bottom line:

- f-strings are fine for experiments
- Prompt Templates are **essential for scalable, reliable LLM applications**

Console-Based Chatbot — Conceptual Flow & Approach

In this setup, we are building a **simple console-based chatbot** that allows continuous interaction between a user and an LLM.

Overall Interaction Flow

The conversation follows a **loop-based dialogue pattern**:

1. The console waits for **user input**
2. The user types a message
3. That message is sent to the **LLM**
4. The LLM generates a response
5. The response is displayed back in the console
6. The system again waits for the next user input

This loop continues, creating a **natural back-and-forth conversation**, similar to chatting with ChatGPT — but entirely inside the terminal.

How the Conversation Feels to the User

From the user's perspective, the console behaves like this:

- User types a message
- AI replies
- User types again
- AI replies again

This pattern repeats until:

- The user explicitly exits, or
- The program is stopped

The experience mimics a **real-time chat**, even though everything is text-based.

Core Concepts Involved

1. Turn-Based Communication

- Each interaction consists of a **user turn** followed by an **AI turn**
- The system strictly alternates between input and output

2. Prompt as User Message

- Whatever the user types is treated as the **prompt**
- That prompt is directly sent to the LLM for generating a response

3. Stateless vs Conversational Context

- In a basic version, each user message is treated independently
- In an advanced version, previous messages can be stored and reused to maintain **conversation context**

4. Continuous Loop

- The chatbot runs inside a loop
 - The loop only stops when a specific exit condition is met (e.g., user types "exit")
-

Why This Approach Is Important

- Helps understand **core LLM interaction mechanics**
- Forms the foundation for:
 - Chatbots
 - Assistants
 - RAG-based conversational systems
 - Agent-based workflows

Almost every advanced chatbot (web, mobile, voice) internally follows the **same logical flow** — only the interface changes.

Key Takeaway

A console-based chatbot is the **simplest and cleanest way** to understand:

- How users interact with LLMs
- How prompts and responses flow
- How conversations are structured programmatically

Once this flow is clear, scaling to web apps, APIs, or full-fledged chat systems becomes much easier.

```
In [8]: from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage
from dotenv import load_dotenv
import os

load_dotenv(override=True)
api_key = os.getenv("OPEN_API_KEY")
model = ChatOpenAI(api_key = api_key)

while True:
    user_input = input('You: ')
    if user_input == 'exit':
        break
    result = model.invoke(user_input)
    print(result.content)
```

Hello! How can I assist you today?

LLM (Large Language Model) and ChatModel are both types of AI models used for natural language processing tasks, but there are some key differences between them.

1. Scope: LLMs are designed to generate text and understand language at a larger scale, while ChatModels are specifically created for conversational interactions with users.
2. Training Data: LLMs are trained on a vast amount of text data from the internet, books, and other sources to learn the nuances of language. ChatModels, on the other hand, are trained on conversational data to understand and respond to user queries.
3. Purpose: LLMs are typically used for tasks like language generation, translation, and text summarization, while ChatModels are used for building chatbots, virtual assistants, and other conversational applications.
4. Interaction: LLMs generate text based on inputs and do not engage in interactive conversations. ChatModels, on the other hand, are designed to engage in real-time conversations with users and respond dynamically to their inputs.

Overall, while both LLMs and ChatModels are used for natural language processing tasks, they serve different purposes and are optimized for different types of interactions with users.

Problem in chatbot

```
In [9]: from langchain_openai import ChatOpenAI
        from langchain_core.messages import SystemMessage, HumanMessage, AIMessage
        from dotenv import load_dotenv
        import os

        load_dotenv(override=True)
        api_key = os.getenv("OPEN_API_KEY")
        model = ChatOpenAI(api_key = api_key)

        while True:
            user_input = input('You: ')
            if user_input == 'exit':
                break
            result = model.invoke(user_input)
            print(result.content)
```

2 is greater than -5.

If the smaller number is 5, subtracting 10 would result in -5.

I apologize, but I need more information to answer your question. Please provide me with the numbers you are referring to so I can determine which one is smaller.

Problem in a Basic Console Chatbot — Lack of Context

When we run our simple console-based chatbot, everything looks fine at first glance. The user asks a question, the AI replies, and the conversation continues.

But there is a **fundamental problem** in this setup.

Observed Issue Through an Example

Step 1: User asks a question

The user asks:

“Tell which one is greater: 2 or 0”

The AI correctly responds that **2 is greater**.

So far, everything is working as expected.

Step 2: User refers to previous information

Now the user says:

“Now multiply the bigger number by 10”

Logically, the correct answer should be:

- Bigger number = 2

- $2 \times 10 = 20$
-

What the AI Actually Does

Instead of using the previous information, the AI responds with something like:

- "Let the bigger number be x, multiplying x by 10 gives 10x"

This response is **technically correct**, but **contextually wrong**.

Why This Problem Happens

Core Reason: No Context Memory

The chatbot:

- Does **not remember** previous user messages
- Does **not know** that "the bigger number" refers to **2**
- Treats every message as a **brand-new, independent prompt**

From the model's perspective:

- The message "*multiply the bigger number by 10*" has no reference point
 - So it answers in a **generic mathematical way**
-

What "Context" Means in LLM Conversations

Context refers to:

- Previous user messages
- Previous AI responses
- The ongoing topic of discussion

In real conversations, humans naturally carry context forward.

But a basic LLM call does **not** do this automatically.

What Is Missing in Our Chatbot

Our current chatbot is:

- **Stateless**
- Each prompt is sent alone
- Previous interactions are not included

As a result:

- Follow-up questions break
 - References like *"that number"*, *"it"*, *"the previous result"* lose meaning
 - Logical continuity is lost
-

Why Context Is Critical for Chatbots

Without context:

- Multi-step reasoning fails
- Conversational flow breaks
- The chatbot behaves like a **Q&A machine**, not a conversational assistant

With context:

- The model understands references
 - Follow-up questions work correctly
 - The chatbot feels intelligent and human-like
-

Key Insight

A chatbot without context is **not truly conversational**.

To build a real chatbot, we must:

- Preserve conversation history
- Send previous messages along with the new prompt
- Allow the LLM to reason across turns

This exact limitation is what leads us to concepts like:

- Chat history
- Message memory
- Conversational chains

Which is the **next logical step** in building robust LLM applications.

```
In [10]: from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage
from dotenv import load_dotenv
import os

load_dotenv(override=True)
api_key = os.getenv("OPEN_API_KEY")
model = ChatOpenAI(api_key = api_key)

chat_history = []

while True:
    user_input = input('You: ')
```

```
chat_history.append(user_input)
if user_input == 'exit' :
    break
result = model.invoke(chat_history)
chat_history.append(result.content)
print("AI: ", result.content)

print(chat_history)
```

AI: -1 is the bigger number when comparing -1 and -2.

AI: If we add 6 to -1, the result would be 5.

AI: Glad I could help! If you have any more questions or need further assistance, feel free to ask.

['what is bigger number in -1 and -2', '-1 is the bigger number when comparing -1 and -2.', 'add 6 in bigger number', 'If we add 6 to -1, the result would be 5.', 'good', 'Glad I could help! If you have any more questions or need further assistance, feel free to ask.', 'exit']

Improving Our Chatbot — Message Roles and Structured Chat History

We successfully built our **first chatbot** and also fixed the **context problem** by storing chat history.

However, **even now the chatbot is not perfect**. There is still a critical design issue.

The New Problem in Our Chatbot

What We Are Currently Doing

- We are storing **all messages** in a single chat history
- The chat history contains multiple messages from the conversation
- But all messages are stored **as plain text**

Example:

- "Hi"
 - "Hello, how can I assist you?"
 - "Tell which one is greater"
 - "-1 is greater than -2"
-

Why This Is a Problem

If someone looks at this chat history:

- There is **no way to know**
 - Which message was sent by the **user**
 - Which message was generated by the **AI**

As the conversation grows:

- The chat history becomes longer
 - Messages get mixed together
 - The LLM itself gets **confused**
-

Impact on the LLM

From the LLM's perspective:

- It cannot differentiate between:
 - Its **own previous responses**
 - The **user's inputs**
- This confusion increases as the chat history grows
- Eventually, future responses may become:
 - Incorrect
 - Illogical
 - Inconsistent

This leads to **conversation breakdown**.

Key Insight

Storing messages alone is not enough.
We must also store *who sent each message*.

What We Ideally Want

Instead of storing messages like this:

- Message 1
- Message 2
- Message 3

We should store them like:

- User → message
- AI → message
- User → message
- AI → message

This preserves:

- Speaker identity
 - Conversation flow
 - Logical continuity
-

The Recommended Approach

A proper chatbot should maintain:

- **Message content**
- **Message role (sender)**

Each message must clearly specify:

- Was it sent by the **user**?
 - Was it sent by the **AI**?
 - Or was it a **system-level instruction**?
-

LangChain's Solution

LangChain already solves this problem in a structured way.

It defines **three explicit message types**:

1. System Message

- Used to set:
 - Behavior
 - Rules
 - Instructions
- Defines *how the AI should behave*

2. Human Message

- Represents:
 - User input
 - User questions
 - User instructions

3. AI Message

- Represents:
 - Model-generated responses
 - AI replies to the user
-

Why This Design Is Powerful

Using structured message roles:

- Keeps conversation **clear**
- Prevents role confusion

- Scales well for long conversations
- Enables reliable multi-turn reasoning

Final Takeaway

A real chatbot is not just about remembering text — it is about remembering **who said what**.

By using **System, Human, and AI messages**, we can build:

- Robust chatbots
- Context-aware assistants
- Production-grade conversational applications

This structured message system is the **foundation of all advanced chatbots** built using LangChain.

```
In [11]: from langchain_core.messages import SystemMessage, HumanMessage, AIMessage
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
import os

load_dotenv(override=True)

# Read the API key
api_key = os.getenv("OPENAI_API_KEY")

model = ChatOpenAI(openai_api_key=api_key, temperature=1.5)

messages=[
    SystemMessage(content='You are a helpful assistant'),
    HumanMessage(content='Tell me about SQL and Pandas')
]

result = model.invoke(messages)

messages.append(AIMessage(content=result.content))

print(messages)
```

```
[SystemMessage(content='You are a helpful assistant', additional_kwargs={}, response_metadata={}), HumanMessage(content='Tell me about SQL and Pandas', additional_kwargs={}, response_metadata={}), AIMessage(content='SQL is a programming language used for managing and manipulating relational databases. With SQL, you can perform various operations such as creating and modifying database tables, querying and retrieving data, inserting or updating records, and managing user permissions.\n\nPandas, on the other hand, is a popular open-source data manipulation and analysis library for Python. It provides powerful data structures such as DataFrame and Series that allow you to easily read, process, filter, and analyze data from various sources such as CSV files, databases, and Excel spreadsheets. Pandas also offers functionality for data cleaning, transformation, and visualization.\n\nIn summary, SQL is used for querying and managing databases, while Pandas is used for data manipulation and analysis in Python programming. They are commonly used together in data science and analytics tasks to extract, process, and analyze data for insights and decision-making.', additional_kwargs={}, response_metadata={})]
```

```
In [12]: # update our chatbot with message

from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage
from dotenv import load_dotenv
import os

load_dotenv(override=True)
api_key = os.getenv("OPEN_API_KEY")
model = ChatOpenAI(api_key = api_key)

chat_history = [
    SystemMessage(content='You are a helpful AI assistant')
]

while True:
    user_input = input('You: ')
    chat_history.append(HumanMessage(content=user_input))
    if user_input == 'exit':
        break
    result = model.invoke(chat_history)
    chat_history.append(AIMessage(content=result.content))
    print("AI: ", result.content)

print(chat_history)
```

AI: AI agents are software programs designed to autonomously perform tasks or make decisions in a specific environment. They work by perceiving their surroundings, processing information, and taking actions to achieve a certain goal. AI agents can range from simple programs that automatically respond to a stimulus to advanced agents that use machine learning algorithms to continuously improve their performance.

Agentic AI refers to AI systems that exhibit characteristics of agency, such as the ability to act autonomously, make decisions, and interact with their environment. Agentic AI is designed to mimic human-like behavior and intelligence, enabling it to perform complex tasks and adapt to changing circumstances.

Overall, AI agents and agentic AI are essential components of artificial intelligence that play a significant role in automating tasks, improving efficiency, and enhancing user experiences in various applications and industries.

```
[SystemMessage(content='You are a helpful AI assistant', additional_kwargs={}, response_metadata={}), HumanMessage(content='tell me about ai agents and agentic ai?', additional_kwargs={}, response_metadata={}), AIMessage(content='AI agents are software programs designed to autonomously perform tasks or make decisions in a specific environment. They work by perceiving their surroundings, processing information, and taking actions to achieve a certain goal. AI agents can range from simple programs that automatically respond to a stimulus to advanced agents that use machine learning algorithms to continuously improve their performance.\n\nAgentic AI refers to AI systems that exhibit characteristics of agency, such as the ability to act autonomously, make decisions, and interact with their environment. Agentic AI is designed to mimic human-like behavior and intelligence, enabling it to perform complex tasks and adapt to changing circumstances.\n\nOverall, AI agents and agentic AI are essential components of artificial intelligence that play a significant role in automating tasks, improving efficiency, and enhancing user experiences in various applications and industries.', additional_kwargs={}, response_metadata={}), HumanMessage(content='exit', additional_kwargs={}, response_metadata={})]
```

Quick Recap — How We Interact with LLMs (Conceptual Overview)

Let's quickly recap **everything we've covered so far**, focusing on the **logical flow and concepts**, not code.

1. How the `invoke()` Function Can Be Used

When working with an LLM, the core interaction happens through the `invoke()` function.

There are **two major ways** to use it.

2. Method 1: Sending a Single Message

What This Means

- You send **one standalone message** to the LLM
- The LLM responds
- The interaction ends

When This Is Used

- One-time queries
- No conversation continuity needed

Examples

- Summarizing a research paper
- Answering a single question

- Generating a paragraph, poem, or explanation
-

3. Static vs Dynamic Single Messages

When sending a **single message**, we learned two approaches:

Static Message

- Prompt is fully hardcoded
- Same prompt every time
- Limited flexibility

Dynamic Message

- Prompt contains placeholders
- Values are filled at runtime
- Much more flexible

Key Tool Used

- **Prompt Template**
 - Allows placeholders + runtime values
 - Gives validation and structure
-

4. Method 2: Sending a List of Messages

Why This Exists

- Real applications need **multi-turn conversations**
- A chatbot must remember:
 - What the user said before
 - What the AI replied before

Example Flow

- User: Hi
- AI: Hello, how can I help?
- User: I need help with homework
- AI: Sure, what subject?

This requires sending a **list of messages**, not just one.

5. Message Types in a Conversation

When using a list of messages, **each message has a role**.

The Three Message Types

1. System Message

- Sets behavior and rules
- Example: "You are a helpful domain expert"

2. Human Message

- Represents user input
- Questions, instructions, follow-ups

3. AI Message

- Model-generated responses
- Added back into chat history

These roles prevent confusion and preserve context.

6. Why Message Roles Are Important

- Prevents mixing user and AI messages
 - Keeps conversation logical
 - Enables long, stable chats
 - Essential for real chatbots
-

7. Static vs Dynamic Messages in Conversations

So far, we learned:

- How to send **static messages** in a list
- How to store **chat history**

But we have **not yet fully explored**:

- **Dynamic messages inside a list of messages**
-

8. The Missing Piece: Dynamic Chat Messages

Just like single prompts can be dynamic, **chat-based messages can also be dynamic**.

Examples

Dynamic System Message

- “You are a helpful **{domain}** expert”
- Domain is decided at runtime

Dynamic Human Message

- “Explain about **{topic}**”
- Topic is provided by the user

Here, **multiple messages** contain placeholders.

9. Why Prompt Template Is Not Enough Here

- `PromptTemplate` works for **single messages**
- But now we are working with:
 - System messages
 - Human messages
 - Multiple placeholders
 - A list of messages

So we need a **specialized tool**.

10. Chat Prompt Template (What Comes Next)

What It Is

- A template system for **lists of messages**
- Supports:
 - System + Human + AI messages
 - Dynamic placeholders
 - Multi-turn conversations

When to Use It

- Whenever:
 - You send a **list of messages**
 - AND those messages need to be **dynamic**
-

Final Mental Model

- **Single message** → Prompt Template
 - **List of messages** → Chat Prompt Template
 - **Static** → Hardcoded text
 - **Dynamic** → Placeholders + runtime values
-

This sets the foundation for:

- Advanced chatbots
- Controlled user experience
- Scalable LLM applications

Next step: **Deep dive into Chat Prompt Templates.**

```
In [13]: ### chat_prompt_template

from langchain_core.prompts import ChatPromptTemplate

chat_template = ChatPromptTemplate([
    ('system', 'You are a helpful {domain} expert'),
    ('human', 'Explain in simple terms, what is {topic}')
])

prompt = chat_template.invoke({'domain': 'Data Science', 'topic': 'Variance'})

print(prompt)
```

```
messages=[SystemMessage(content='You are a helpful Data Science expert', additional_kwargs={}, response_metadata={}), HumanMessage(content='Explain in simple terms, what is Variance', additional_kwargs={}, response_metadata={})]
```

Message Placeholder — Conceptual Understanding

This plays a **very important role in real-world chatbot systems.**

What Is a Message Placeholder?

A **Message Placeholder** in LangChain is a **special placeholder used inside a Chat Prompt Template** that allows you to:

- Dynamically insert **chat history**
- Insert a **list of past messages**
- Do this **at runtime**, not hardcoded

In simple words:

Message Placeholder is used when you want to inject an entire conversation history into a prompt dynamically.

Why Do We Need Message Placeholders?

So far, we have learned:

- How to send a **list of messages** to an LLM
- How to make those messages **dynamic** using Chat Prompt Templates

But there is still a gap.

The Problem

In real applications:

- Conversations do not always happen in one session
- Users often come back **after some time**
- The chatbot must remember **past interactions**

If the chatbot does not know what happened earlier, it will:

- Lose context
 - Give incorrect or irrelevant answers
-

Real-World Scenario Example

Imagine a **customer support chatbot** for an e-commerce company.

Day 1

- User: I want a refund for my order
- Bot: Your refund has been initiated. You will receive it in 3–5 business days

This conversation is saved in a database.

Day 3

- User returns and asks:
"What is the status of my refund?"

Now think:

- This is a **new chat session**
- But the question depends entirely on the **previous conversation**

Without loading past messages:

- The chatbot has no idea which refund the user is talking about
-

The Core Requirement

To answer correctly, the chatbot must:

- Load the **previous chat history**

- Inject it back into the prompt
- Continue the conversation naturally

This is exactly where **Message Placeholder** is used.

How Message Placeholder Helps (Conceptually)

Instead of manually inserting:

- Every old user message
- Every old AI reply

You do the following conceptually:

- Maintain chat history in a database
- Load it when a new session starts
- Insert the entire history at once using a **Message Placeholder**

This keeps the prompt:

- Clean
 - Scalable
 - Easy to manage
-

How It Fits Into Chat Prompt Templates

Think of a Chat Prompt Template as:

- System message (rules / behavior)
- Message Placeholder (entire past conversation)
- Current user message

The Message Placeholder acts as a **dynamic slot** where:

- Any number of past messages can be injected
 - Order and roles (user / AI) are preserved
-

Key Benefits of Message Placeholders

- Enables **context-aware chatbots**
- Supports **multi-session conversations**
- Prevents repetition and confusion
- Essential for:
 - Customer support bots
 - Assistants with memory
 - Long-running conversations

Mental Model to Remember

- **Prompt Template** → Dynamic single message
 - **Chat Prompt Template** → Dynamic list of messages
 - **Message Placeholder** → Dynamic insertion of entire chat history
-

Final Takeaway

If you are building:

- A serious chatbot
- A support assistant
- A conversational AI with memory

Then **Message Placeholders are not optional** — they are a core requirement.

This completes the foundational understanding of:

- Prompts
- Templates
- Chat history
- Context management in LangChain

```
In [14]: from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

# chat template
chat_template = ChatPromptTemplate([
    ('system', 'You are a helpful customer support agent'),
    MessagesPlaceholder(variable_name='chat_history'),
    ('human', '{query}')
])

chat_history = []
# Load chat history
with open('chat_history.txt') as f:
    chat_history.extend(f.readlines())

print(chat_history)

# create prompt
prompt = chat_template.invoke({'chat_history': chat_history, 'query': 'Where is my
print(prompt)
```

```
[ 'HumanMessage(content="I want to request a refund for my order #12345.")\n', 'AI\nMessage(content="Your refund request for order #12345 has been initiated. It will\nbe processed in 3-5 business days.")']\nmessages=[SystemMessage(content='You are a helpful customer support agent', addit\nional_kwargs={}, response_metadata={}), HumanMessage(content='HumanMessage(conten\nt="I want to request a refund for my order #12345.")\n', additional_kwargs={}, re\nsponse_metadata={}), HumanMessage(content='AIMessage(content="Your refund request\nfor order #12345 has been initiated. It will be processed in 3-5 business day\ns.")', additional_kwargs={}, response_metadata={}), HumanMessage(content='Where i\ns my refund', additional_kwargs={}, response_metadata={})]
```

Prompt Engineering Techniques — Structured Notes (Single Cell)

These notes explain **different prompt engineering techniques** used while interacting with Large Language Models (LLMs).

Each technique includes:

- **What it is**
 - **Why it is used**
 - **Simple examples (text-only, no code)**
-

1. Zero-Shot Prompting

What is it?

Zero-shot prompting means **asking the model a question without giving any examples**.

The model relies purely on its pre-trained knowledge.

When to use?

- When the task is **simple or common**
- When you want **quick answers**
- When no task-specific guidance is needed

Example

Prompt:

Explain what overfitting is in machine learning.

Model behavior:

The model directly explains the concept without seeing any example.

2. One-Shot Prompting

What is it?

One-shot prompting provides **exactly one example** before asking the actual task.

Why it helps?

- Gives the model **task clarity**
- Reduces ambiguity compared to zero-shot

Example

Prompt:

Example:
Input: The movie was boring and slow
Output: Negative

Now classify:
Input: The food was amazing
Output:

Expected output:

Positive

3. Few-Shot Prompting

What is it?

Few-shot prompting provides **multiple examples** to teach the model the expected pattern.

When to use?

- When task format is important
- When zero-shot output is inconsistent
- When you want higher accuracy

Example

Prompt:

Input: I love this phone
Output: Positive

Input: This is the worst service ever
Output: Negative

Input: The product is okay

Output:

Expected output:

Neutral

4. Chain-of-Thought (CoT) Prompting

What is it?

Chain-of-Thought prompting encourages the model to **think step-by-step** before answering.

Why is it powerful?

- Improves reasoning
- Reduces logical errors
- Useful for math, logic, and multi-step problems

Example

Prompt:

Solve step by step:

If a train travels 60 km in 1 hour, how far will it travel in 2.5 hours?

Model reasoning (internally):

- Speed = 60 km/hr
- Time = 2.5 hr
- Distance = speed × time

Final Answer:

150 km

5. Explicit Chain-of-Thought Prompting

What is it?

You explicitly **ask the model to show its reasoning**.

Example

Prompt:

Explain your reasoning step by step and then give the final answer:

What is 15% of 200?

Expected behavior:

- Step 1: Convert 15% to decimal $\rightarrow 0.15$
 - Step 2: Multiply $\rightarrow 0.15 \times 200$
 - Final Answer: 30
-

6. Self-Consistency Prompting

What is it?

The model generates **multiple reasoning paths** and then selects the most consistent answer.

Why use it?

- Reduces hallucinations
- Improves complex reasoning accuracy

Example

Prompt idea:

Solve the problem using different reasoning paths and give the most consistent answer:
What is the square root of 144?

Final Answer:

12

7. Role-Based Prompting

What is it?

You assign a **specific role or persona** to the model.

Why it works?

- Sets context
- Improves tone, depth, and relevance

Example

Prompt:

You are a senior data scientist.
Explain bias-variance tradeoff to a beginner.

Effect:

The explanation becomes simpler, structured, and beginner-friendly.

8. Instruction Prompting

What is it?

You give **clear, direct instructions** about what the model should do.

Example

Prompt:

Summarize the following text in 3 bullet points using simple language.

Effect:

The output strictly follows format and constraints.

9. Prompt Chaining

What is it?

Breaking a complex task into **multiple smaller prompts**, where output of one becomes input to another.

Why use it?

- Better control
- Improved accuracy
- Easier debugging

Example (Conceptual Flow)

1. First prompt: Extract key points from an article
 2. Second prompt: Convert key points into a summary
 3. Third prompt: Rewrite summary in simple language
-

10. ReAct Prompting (Reason + Act)

What is it?

ReAct combines:

- **Reasoning** (thinking)
- **Actions** (tool usage or decisions)

Where is it used?

- Agents
- Tool-using LLMs
- Autonomous workflows

Example

Prompt behavior (conceptual):

- Think: "I need population data"
 - Act: "Search population of India"
 - Think: "Now calculate density"
 - Answer with reasoning + result
-

11. Contextual Prompting

What is it?

Providing **background information** before asking the question.

Example

Prompt:

Context: This chatbot is used for customer support in an e-commerce platform.

Question: How would you respond to a delayed delivery complaint?

Effect:

More realistic and domain-specific response.

12. Constraint-Based Prompting

What is it?

You restrict the output using **rules or boundaries**.

Example

Prompt:

Answer in less than 50 words and avoid technical jargon:
What is machine learning?

Final Mental Model

- **Simple task** → **Zero-shot**
 - **Pattern-based task** → **Few-shot**
 - **Reasoning task** → **Chain-of-Thought**
 - **Domain clarity** → **Role-based**
 - **Complex workflows** → **Prompt chaining / ReAct**
-

Key Takeaway

Prompt engineering is not about tricks.

It is about **clear communication with the model**.

A better prompt → better reasoning → better output.

In []: